



Conformance Checking: Foundations, Milestones and Challenges

Josep Carmona¹, Boudewijn van Dongen^{2(✉)}, and Matthias Weidlich³

¹ Universitat Politècnica de Catalunya, Barcelona, Spain

² Eindhoven University of Technology, Eindhoven, The Netherlands
B.F.v.Dongen@tue.nl

³ Humboldt-Universität zu Berlin, Berlin, Germany

Abstract. By relating observed and modelled behaviour, conformance checking unleashes the full power of process mining. Techniques from this discipline enable the analysis of the quality of a process model discovered from event data, the identification of potential deviations, and the projection of real traces onto process models. This way, the insights gained from the available event data can be transferred to a richer conceptual level, amenable for a human interpretation. The aforementioned functionalities are grounded on the use of conformance checking artefacts that explicit the relation between observed and modelled behaviour. This chapter describes these artefacts, and builds upon them to gain evidence-based insights on the processes of an organization. Moreover, we overview the applications of conformance checking and propose a general framework that incorporates these applications. Finally, milestones and challenges of the field are outlined.

1 Introduction

Organisations tend to define, by means of conceptual models, complex business processes that must be followed to achieve their objectives [22]. Sometimes the corresponding processes are distributed in different systems, and most of the cases include human tasks, enabling the occurrence of unexpected deviations with respect to the (normative) process model. This is aggravated by the appearance of more and more complex processes, where the observations are provided by heterogeneous sources, such as Internet-of-Things (IoT) devices involved in Cyber-physical Systems [46].

Conformance checking techniques provide mechanisms to relate modelled and observed behaviour, so the frictions between the footprints left by process executions, and the process models that formalise the expected behaviour, can be revealed [14]. As it has been already commented in the first chapters of this book, process executions are often materialized and stored by means of event logs. Table 1 shows an example of an event log for a loan application process.

Conformance checking is expected to be the fastest growing segment in process mining for the next years¹. The main reason for this forthcoming industrial interest is the promise of having event data and process models aligned, thus increasing the value of process models within organizations.

Given an event log and a process model, conformance checking techniques yield some explicit description of their consistent and deviating parts, here referred to as a *conformance artefact*. In the first part of this chapter, we focus on three main conformance artefacts that are covering most of the spectrum of conformance checking:

- Behavioural rules such as ordering constraints for activities imposed by the model that are violated by some traces of the event log;
- Events of traces that could correctly be replayed by task executions in the process model, or for which the replay failed;
- An alignment between the events of a trace of the event log and the task executions of an execution sequence of the model.

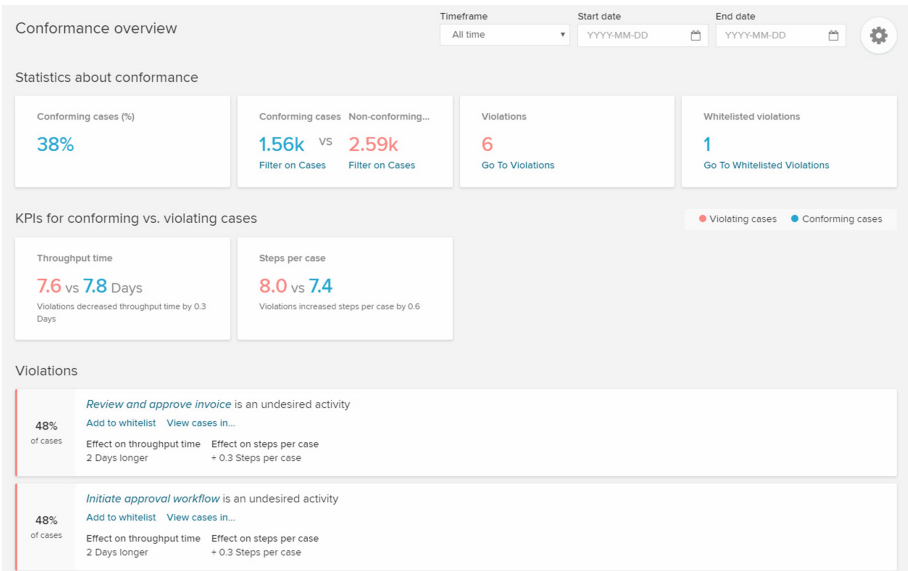


Fig. 1. Example of conformance checking in Celonis.

Remarkably, a conformance artefact enables conclusions on the relation between the event log and the process model. By interpreting the conformance artefact, for instance, the fitness and precision of the model regarding the given log is quantified. Such an interpretation may further involve decisions on how to weight and how to attribute any encountered deviation (see the end of this chapter

¹ <https://www.marketsandmarkets.com/Market-Reports/process-analytics-market-254139591.html>.

for a discussion on this topic). Since the log and the model are solely representations of the process, both of them may differ in how they abstract the process.

Differences in the representations of a process may, of course, be due to inaccuracies. For example, an event log may be recorded by an erroneous logging mechanism (see next chapter of this handbook for understanding this in depth), whereas a process model may be outdated. Yet, differences may also be due to different purposes and constraints that guide how the process is abstracted and therefore originate from the pragmatics of the respective representation of the process. Think of a logging mechanism that does not track the execution of a specific activity due to privacy considerations, or a model that outlines only the main flow of the process to clarify its high-level phases. Either way, the respective representations are not *wrong*, but differ because of their purpose and the constraints under which they have been derived.

By linking an event log and a process model through a conformance artefact, the understanding of the underlying process can be improved. That includes techniques for process enhancement (see [18]). For instance, traces of an event log can be replayed in the process model, while taking into account the deviations between the log and model as materialised in the conformance artefact. Commercial tools that include conformance checking nicely display these deviations on their dashboards, as can be seen in Fig. 1. Another example includes the inspection of the conditions that govern the decision points in a process. The conformance artefact can be used to derive a classification problem per decision point, which enables discovery of the respective branching conditions. Assuming that the model represents the desired behaviour of the process, the conformance artefact further enables conclusions on how the current realisation of the process needs to be adapted.

There exist different algorithmic perspectives to relate modelled and observed behaviour: rule checking, token-replay and alignments.

A process model defines a set of tasks along with causal dependencies for their execution. As such, a process model constrains the possible behaviour of a process in terms of its execution sequences. Instead of considering the set of possible execution sequences of a process model, however, the basic idea of rule-based conformance checking is to exploit rules that are satisfied by all these sequences as the basis for analysis. Such rules define a set of constraints that are imposed by the process model. Verification of these constraints with respect to the traces of an event log, therefore, enables the identification of conformance issues.

Unlike rule checking that is grounded in information derived from the process model, token replay takes the event log as the starting point for conformance analysis. As indicated already by its name, this technique replays each trace of the event log in the process model by executing tasks according to the order of the respective events. By observing the states of the process model during the replay, it can be determined whether, and to what extent, the trace indeed corresponds to a valid execution sequence of the model.

In spite of the two aforementioned class of techniques to relate modelled and observed behaviour, most conformance checking techniques rely on the notion of alignment [1]: given an observed trace σ , query the model to obtain the execution sequence γ that is most similar to σ . The computation of alignments is

a computational challenge, since it encompasses the exploration of the model state space, an object that is worst-case exponential with respect to the size of the model or the trace.

Table 1. Example of a log of the loan application process.

Event	Application	Offer	Activity	Amount	Signed	Timestamp
...	...	...	...	...	...	...
e_{13}	A5634		Application submitted	€2,000		Jan 01, 12:31
e_{14}	A5634		Accept application	€2,000		Jan 01, 12:32
e_{15}	A5635		Application submitted	€5,000		Jan 02, 04:31
e_{16}	A5635		Accept application	€5,000		Jan 02, 04:32
e_{17}	A5636		Application submitted	€200		Jan 03, 06:59
e_{18}	A5636		Accept application	€200		Jan 03, 07:00
...	...	...	...	...	...	...
e_{22}	A5634		Finalise application			Jan 03, 09:00
e_{23}	A5636		Finalise application			Jan 03, 09:01
e_{24}	A5635		Decline application			Jan 03, 09:02
e_{25}	A5635		Decline application			Jan 03, 09:03
...	...	...	...	...	...	...
e_{30}	A5636	O3521	Select and send offer	€500		Jan 04, 16:32
...	...	...	...	...	...	...
e_{37}	A5634	O3541	Select and send offer	€1,500		Jan 05, 12:32
e_{38}	A5636	O3521	Receive offer		NO	Jan 05, 12:33
e_{38}	A5636	O3521	Cancel offer			Jan 05, 12:34
e_{39}	A5636	O3542	Select and send offer	€500		Jan 05, 13:29
e_{40}	A5636	O3542	Receive offer		YES	Jan 08, 08:33
e_{41}	A5636	O3542	Accept offer			Jan 08, 16:34
e_{42}	A5634	O3541	Receive offer		NO	Jan 10, 10:00
...	...	...	...	...	...	...
e_{54}	A5634	O3541	Decline offer			Jan 10, 10:04
...	...	...	...	...	...	...
e_{64}	A5634		Decline application			Jan 10, 10:05
e_{65}	A5634		Application finished			Jan 10, 10:06
e_{66}	A5636		Approve and activate application			Jan 10, 10:07
e_{67}	A5636		Application finished			Jan 10, 10:08
...	...	...	...	...	...	...

Once conformance artefacts are computed, the next natural step is to use them. The main applications arising from these artefacts are listed in this chapter as well, as a gentle introduction to some of the chapters devoted to this in this book. We highlight performance analysis and decision point analysis as natural examples of the application of conformance checking.

Furthermore, depending on the trust we put on the two main elements (trust on the log, trust on the model), conformance checking can be generalized as a framework that unifies diverse analysis techniques in the field of process mining [48]. As such, this framework includes several instantiations already known to the reader.

We finish the chapter by listing important milestones and challenges, some of them being already under consideration by the research community, like the computational feasibility of the underlying techniques.

2 Relating Observed and Modelled Behaviour: The Basics

In this section, we discuss the basic notions and techniques to relate observed and modelled behaviour. To this end, we first review generic quality dimensions on this relation (Sect. 2.1). Subsequently, we turn to three different types of conformance checking artefacts that capture the relation between a trace observed in the event log and a process model, namely artefacts grounded in rule checking (Sect. 2.2), token replay (Sect. 2.3), and alignments (Sect. 2.4), see also Fig. 2. A detailed explanation of the contents of this section can be found in [14].

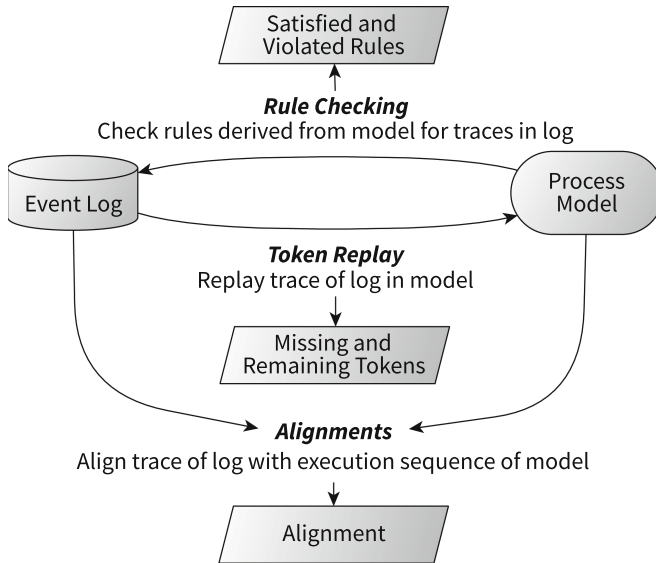


Fig. 2. General approaches to conformance checking and resulting conformance artefacts (from [14]): rule checking, token replay, and alignments. All techniques take a trace of an event log and a process model as input. However, conceptually, rule checking starts from the behaviour of the process model, extracting constraints to check for a trace. Token replay, in turn, starts from the behaviour of a single trace, trying to replay the trace in a process model. Alignments, in turn, adopt an inherently symmetric view.

2.1 Quality Dimensions to Relate Process Models and Event Logs

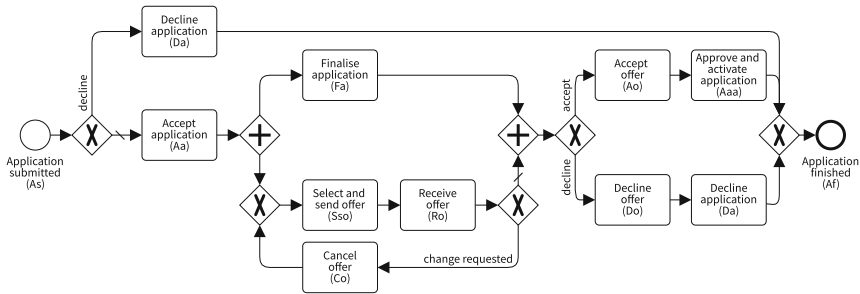


Fig. 3. Example process model of a loan application process in BPMN.

By relating observed and modelled behaviour, an organization can get insights on the execution of their processes with respect to the expectations as described in the models. If both process model M and event log L are considered as languages, their relation can be used to measure how good is a process model in describing the behaviour recorded in an event log.

Hence, confronting M and L can help into understanding the complicate relation between modelled and recorded behaviour. We now provide two views on this relation that represent two alternative perspectives: *fitness* and *precision*. To illustrate this, in this chapter we will be using a process for a loan application. A process model illustrating this process is described in Fig. 3. According to this model, a submitted application is either accepted or rejected, depending on the applicant's data. An accepted application is finalised by a worker, in parallel with the offer process. For each application, an offer is selected and sent to the customer. The customer reviews the offer and sends it back. If the offer is accepted, the process continues with the approval of the application and the activation of the loan. If the customer declines the offer, the application is also declined and the process ends. However, the customer can also request a new offer, in which case the offer is cancelled and a new offer is sent to the customer.

Fitness measures the ability of a model to explain the recorded execution of a process as recorded in an event log (see the example of Fig. 4 for an example of fitting behaviour). It is the main measure to assess whether a model is well-suited to explain the recorded behaviour. To explain a certain trace, the process model is queried to assess its ability in replaying the trace, taking into account the control flow logic expressed in the model.

In general, fitness is the fraction of the behaviour of the log that is also allowed by the model. It can be expressed as follows.

$$fitness = \frac{|L \cap M|}{|L|} \quad (1)$$

Let us have a look at this fraction in more detail by examining the extreme cases. Fitness is 1, if the entire behaviour that we see in the log L is covered by the model M . Conversely, fitness is 0, if no behaviour in the log L is captured by the model M . In the remainder of this section, we will describe three different algorithms deriving artefacts that can be used to evaluate fitness.

We define a trace to be either *fitting* (it corresponds to an execution sequence of the model) or *non-fitting* (there is some deviation with respect to all execution sequences of the model). For instance, the trace corresponding case A5634 in our running example is fitting, since there is an execution sequence of the model that perfectly reproduces this case, as shown in Fig. 4. In contrast, Fig. 5 shows the information for a trace that does not contain the event to signal that the application has been finalised (Fa).

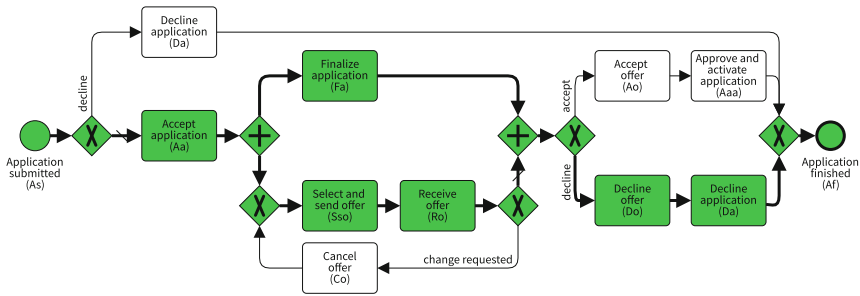


Fig. 4. Loan application process model with highlighted path corresponding to the fitting trace $\langle As, Aa, Fa, Sso, Ro, Do, Da, Af \rangle$ of case A5634 from the event log of Table 1.

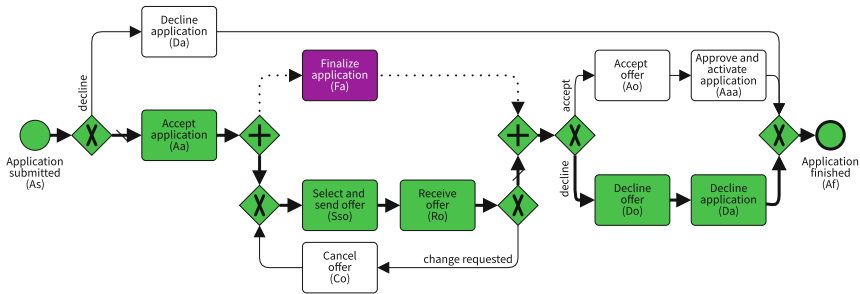


Fig. 5. Loan application process model with highlighted path corresponding to a trace $\langle As, Aa, Sso, Ro, Do, Da, Af \rangle$, which does not include an event to signal that the application has been finalised (Fa). In magenta, we show that the task (Fa) has not been observed, but it is required to reach the final state of the process model.

Precision is the counterpart of fitness. It can be calculated by looking at the fraction of the model behaviour that is covered in the log.

$$precision = \frac{|L \cap M|}{|M|} \quad (2)$$

We see that precision shares the numerator in the fraction with fitness from (1). This implies that if we have a log and a model with no shared behaviour, fitness is zero, and by definition also precision is zero. However, the denominator is replaced with the amount of modelled behaviour.

In summary, for the two main metrics reported above, algorithms that can assess the relation between log and model need to be considered. In the next section, we describe the three main algorithmic perspectives to accomplish this task. For an extensive analysis of metrics to assess the relation between observed and modelled behaviour, including metrics like *generalization* or *simplicity*, the reader is referred to [14]. Intuitively, generalization complements precision by quantifying the amount of behaviour that is modelled in a process model, but not observed in an event log. In practice, an event log cannot be expected to be complete, i.e., to contain all possible process behaviour (e.g., all possible interleavings of concurrent activities or all possible numbers of iterations of repetitive behaviour). Hence, a process model is typically assumed to generalize to some extent, i.e., not to show perfect precision, and generalization measure aim to quantify this amount of imprecision. Simplicity, in turn, refers to the structure and complexity of the model. Intuitively, simplicity measures induce some preference for process models that behave similarly in terms of the other dimensions, with the argument being that simple models are generally to be preferred.

2.2 Rule Checking

The basic idea of rule-based conformance checking is to exploit rules that are satisfied by all the execution sequences of a process model as the basis for analysis. Such rules define a set of constraints that are imposed by the process model. The verification of these constraints with respect to the traces of an event log, therefore, enables the identification of conformance issues.

Considering the running example of our loan application process as depicted in Fig. 3, rules derived from the process model include:

- R1: An application can be accepted (*Aa*) at most once.
- R2: An accepted application (*Aa*), that must have been submitted (*As*) earlier, and eventually an offer needs to be selected and sent (*Sso*) for it.
- R3: An application must never be finalised (*Fa*), if the respective offer has been declined (*Do*) already.
- R4: An offer is either accepted (*Ao*) or declined (*Do*), but cannot be both accepted and declined.

A careful inspection of each one of the rules above would reveal that they are different in nature: rule R1 is an example of *cardinality rule*, which defines

an upper and lower bound for the number of executions of an activity. Rule R2 contains a *precedence rule*, which establishes that the execution of a certain activity is preceded by at least on execution of another activity. Rule R3 establishes an *ordering rule*, whereas rule R4 represents an *exclusiveness rule*. Tables 2 and 3 show examples of cardinality and exclusiveness rules, respectively, for the running example and two log traces.

Table 2. Precedence rules derived for the process model of the running example and their satisfaction (✓) and violation (✗) by the exemplary log trace $\langle As, Sso, Fa, Ro, Co, Ro, Aaa, Af \rangle$. Each non-empty cell refers to a precedence rule. For instance, the activity to finalize the application (*Fa*) is preceded by the submission of the application (*As*) and the acceptance of the application (*Aa*). Yet, only the former rule is satisfied, whereas the latter one is violated in the given trace.

	<i>As</i>	<i>Da</i>	<i>Aa</i>	<i>Fa</i>	<i>Sso</i>	<i>Ro</i>	<i>Co</i>	<i>Ao</i>	<i>Aaa</i>	<i>Do</i>	<i>Af</i>
<i>As</i>											
<i>Da</i>	✓										
<i>Aa</i>	✓										
<i>Fa</i>	✓		✗								
<i>Sso</i>	✓		✗								
<i>Ro</i>	✓		✗		✓						
<i>Co</i>	✓		✗		✓	✓					
<i>Ao</i>	✓		✓	✓	✓	✓					
<i>Aaa</i>	✓		✗	✓	✓	✓		✗			
<i>Do</i>	✓		✓	✓	✓	✓					
<i>Af</i>	✓										

By assessing to what extent the traces of a log satisfy the rules derived from a process model, rule-based conformance checking focuses on the fitness dimension, i.e., the ability of the model to explain the recorded behaviour. Traces are fitting, if they satisfy the rules, or non-fitting if that is not the case. Let R_M be a predefined set of rules. Fitness can be defined according² to R_M :

$$\text{fitness}(L, M) = \frac{|\{r \in R_M \mid r \text{ is satisfied by all } t \in L\}|}{|R_M|} \quad (3)$$

As the reader may already have grasped, the dimension of precision is not targeted by rule-checking.

² Notice that this makes fitness to depend on a particular set of rules, which is a limitation of the rule-based fitness checking.

Table 3. Exclusiveness rules derived for the process model of the running example and their satisfaction (✓) and violation (✗) by the exemplary log trace $\langle As, Aa, Sso, Ro, Fa, Ao, Do, Da, Af \rangle$. Again, each non-empty cell denotes a rule, i.e., the absence of the execution of two activities for the same case. For instance, the acceptance of an offer (*Ao*) must not be executed for cases for which the application is declined (*Da*). Yet, in the given trace, respective events for both activities can be found, so that the rule is marked as being violated.

	<i>As</i>	<i>Da</i>	<i>Aa</i>	<i>Fa</i>	<i>Sso</i>	<i>Ro</i>	<i>Co</i>	<i>Ao</i>	<i>Aaa</i>	<i>Do</i>	<i>Af</i>
<i>As</i>	✓										
<i>Da</i>		✓						✗		✓	
<i>Aa</i>			✓								
<i>Fa</i>				✓							
<i>Sso</i>											
<i>Ro</i>											
<i>Co</i>											
<i>Ao</i>		✗						✓		✗	
<i>Aaa</i>		✓							✓	✓	
<i>Do</i>								✗	✓	✓	
<i>Af</i>											✓

2.3 Token Replay

Intuitively, this technique replays each trace of the event log in the process model by executing tasks according to the order of the respective events. By observing the states³ of the process model during the replay, one can determine whether, and to what extent, the trace indeed corresponds to a valid execution sequence of the process model.

In essence, token replay postulates that each trace in the event log corresponds to a valid execution sequence of the process model. This is verified by step-wise executing tasks of the process model, according to the order of the respective events in the trace. During this replay, we may observe two cases that hint at non-conformance (see Fig. 6):

- (i) the execution of a task requires the consumption of a token on the incoming arc, but the arc is not assigned any token in the current state, i.e., a token is *missing* during replay;
- (ii) the execution of a task produces a token at an outgoing arc, but this token is not consumed eventually, i.e., a token is *remaining* after replay.

³ A state of a BPMN model is a distribution of tokens over the control flow arcs. A task is enabled in a state if its incoming control flow arc is assigned a token by the respective distribution. If it executes, this token is *consumed*, i.e., no longer assigned to the arc. Moreover, a token is *produced* on the outgoing control flow arc of the task.

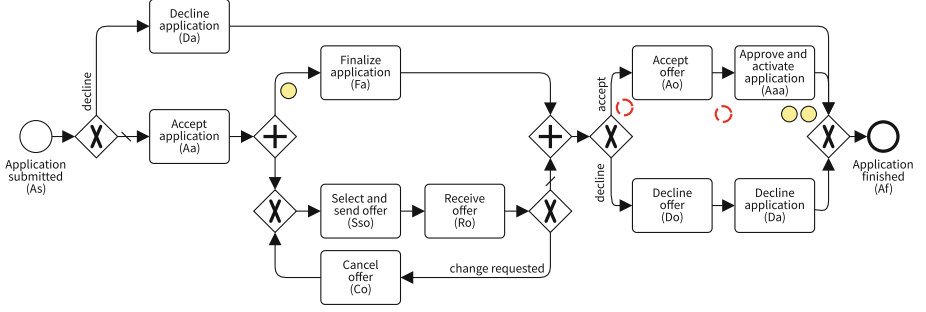


Fig. 6. State reached after replaying the full trace $\langle As, Aa, Sso, Ro, Ao, Aaa, Aaa \rangle$. One can see that there are three remaining tokens (denoted by yellow background), and two missing tokens (denoted by dashed red lines). (Color figure online)

By exploring whether the replay of a trace yields missing or remaining tokens, replay-based conformance checking mainly focuses on the fitness dimension. That is, the ability of the model to explain the recorded behaviour is the primary concern. Traces are fitting if their replay does not yield any missing or remaining tokens, and non-fitting otherwise:

$$\text{fitness}(L, M) = \frac{1}{2} \left(1 - \frac{\sum_{t \in L} \text{missing}(t, M)}{\sum_{t \in L} \text{consumed}(t, M)} \right) + \frac{1}{2} \left(1 - \frac{\sum_{t \in L} \text{remaining}(t, M)}{\sum_{t \in L} \text{produced}(t, M)} \right) \quad (4)$$

In contrast to rule checking, precision can be estimated using token replay [34], but unfortunately, the corresponding technique strongly relies on the assumption that traces are fitting; if they are not, then the estimation of precision through token replay can be significantly degraded [2].

2.4 Alignments

Alignments take a symmetric view on the relation between modelled and recorded behaviour. Specifically, they can be seen as an evolution of token replay. Instead of establishing a link between a trace and sequences of task executions in the model through replay, alignments directly connect a trace with an execution sequence of the model.

An alignment connects a trace of the event log with an execution sequence of the process model. It is represented by a two-row matrix, where the first row consists of activities as their execution is signalled by the events of the trace and a special symbol $\gg$ (jointly denoted by e_i below), and the second row consists of the activities that are captured by task executions of an execution sequence of the process model and a special symbol $\gg$ (jointly denoted by a_i):

log trace	e_1	e_2	$\dots$	e_n
execution sequence	a_1	a_2	$\dots$	a_m

Each column in this matrix, a pair (e_i, a_i) , is a *move* of the alignment, meaning that an alignment can also be understood as a sequence of moves. There are different types of such moves, each encoding a different situation that can be encountered when comparing modelled and recorded behaviour. We consider three types of moves:

- *Synchronous move*: A step in which the event of the trace and the task in the execution sequence correspond to each other. Synchronous moves denote the expected situation that the recorded events in the trace are in line with the tasks of an execution sequence of the process model. In the above model, a synchronous move means that it holds $e_i = a_i$ and $e_i \neq \gg$ (and thus $a_i \neq \gg$).
- *Model move*: When a task should have been executed according to the model, but there is no related event in the trace, we refer to this situation as a model move. As such, the move represents a deviation between the trace and the execution sequence of the process model in the sense that the execution of an activity has been skipped. In the above model, a model move is denoted by a pair (e_i, a_i) with $e_i = \gg$ and $a_i \neq \gg$.
- *Log move*: When an event in the trace indicates that an activity has been executed, even though it should not have been executed according to the model, the alignment contains a log move. Being the counterpart of a model move, a log move also represents a deviation in the sense of a superfluous execution of an activity. A log move is denoted by a pair (e_i, a_i) with $e_i \neq \gg$ and $a_i = \gg$.

Alignments are constructed only from these three types of moves (see an in-depth explanation on this in [14]). For instance, let us use the running example (see Fig. 3) and the trace $\langle As, Aa, Sso, Ro, Ao, Aaa, Aaa \rangle$. A possible alignment with this trace is:

log trace	As	Aa	Sso	Ro	$\gg$	Ao	Aaa	Aaa	$\gg$
execution sequence	As	Aa	Sso	Ro	Fa	Ao	Aaa	$\gg$	Af

This alignment comprises six synchronous moves, one log move, $(Aaa, \gg)$, and two model moves, $(\gg, Fa)$ and $(\gg, Af)$. The log move $(Aaa, \gg)$ indicates that the application had been approved and activated, even though this was not expected in the current state of processing (as this had just been done). The model move $(\gg, Fa)$ is the situation of the process model requiring that the application be finalised, which has not been done according to the trace. Furthermore, one can easily extract the original trace by projecting away the special symbol for skipping from the top row. Applying the projection to the bottom row yields the execution sequence of the model $(\langle As, Aa, Sso, Ro, Fa, Ao, Aaa, Af \rangle)$.

In general, *optimal alignments*, i.e., alignments with a minimal number of model or log moves, are preferred. The alignment shown above is optimal since there is no other alignment with least number of deviations. Computing (optimal) alignments is a hot research topic, which has been addressed in many papers in the last years [1, 9, 19, 31, 39, 44, 45, 56–58, 60, 66, 68]. In this paper, however, we

will refrain from describing the state-of-the-art methods for alignment computation, and refer the interested reader to the aforementioned papers, or to [14].

Moreover, the optimality of alignments may also be generalized in terms of a cost function that assigns costs to particular model moves or log moves, thereby enabling the categorization of deviations in terms of their severity. Then, an alignment is optimal, if the sum of costs assigned to all its moves is minimal. Setting the cost for all model moves and log moves to one, and for synchronous moves to zero, yields the aforementioned notion of optimality, i.e., alignments with a minimal number of model or log moves.

Remarkably, alignments provide a simple means to quantify fitness. Again, this may be done based on the level of an individual trace or the event log as a whole. However, the aggregated cost of log moves and model moves may be a misleading measure, though, as it is not normalised. A common approach, therefore, is to normalise this cost by dividing it by the worst-case cost of a aligning the trace with the given model. Under a uniform assignment of costs to log and model moves, such a worst-case cost originates from an alignment in which each event of the trace T_i relates to a log move, whereas all task executions of a sequence σ of the model relate to a model move and σ is as short as possible. Since the cost induced by the model moves of an execution sequence depends on its length, the shortest possible execution sequence leading from the initial state to a final state in the model is considered for this purpose.

Realising the above idea, we obtain two ratios that denote the relative share of non-fitness in the alignments of a trace or an event log, respectively. Let M be a model and L an event log. Then, we denote by $\text{cost}(t, M)$ the cost of an optimal alignment of a trace $t \in L$ with respect to the model. Furthermore, let $\text{cost}(t, \langle \rangle)$ and $\text{cost}(\langle \rangle, x)$ be the costs of aligning a trace t with an empty execution sequence, or some execution sequence $x \in M$ of the model with an empty trace, respectively. Then, fitness based on alignments is quantified for a trace or an event log:

$$\text{fitness}(L, M) = 1 - \left(\frac{\sum_{t \in L} \text{cost}(t, M)}{\sum_{t \in L} (\text{cost}(t, \langle \rangle)) + |L| \times \min_{x \in M} \text{cost}(\langle \rangle, x)} \right) \quad (5)$$

A simple precision metric based on alignments is grounded in the general idea of *escaping edges* [34]. To give the intuition, we assume that (i) the event log fits the process model; and (ii) that the process model is deterministic. The former means that we simply exclude non-fitting traces, for which the optimal alignment contains log moves or model moves, from the assessment of the precision of the model. The latter refers to a process model not being able to reach a state, in which two tasks that capture the same activity of the process are enabled. The model of our running example (see Fig. 3) is deterministic.

For the activity of each event of a trace of the event log, we can determine a state of the process model right before the respective task would be executed. Under the above assumptions, this state is uniquely characterised. What is relevant when assessing precision, is the number of tasks enabled in this state of

the process model. Let M be a process model and L an event log, with $t \in L$ as a trace and, overloading notation, $e \in t$ as one of the events of the trace. Then, by $enabled_M(e)$, we denote the number of tasks and, due to determinism of the process model also the number of activities that can be executed in the state right before executing the task corresponding to e .

Similarly, we consider all traces of the log that also contain events related to the activity of event e , say a , and have the same prefix, i.e., events that indicate that the same sequence of activities has been executed before an event signalling the execution of activity a . Then, we determine the number of activities for which events signal the execution directly after this prefix, i.e., the set of activities that have been executed in the same context as the activity a as indicated by event e . Let this number of activities be denoted by $enabled_L(e)$, which, under the above assumptions, is necessarily less than or equal to $enabled_M(e)$. Then, the ratio of both numbers captures the amount of ‘escaping edges’ that represent modelled behaviour that has not been recorded. As such, precision of log L and M is quantified as follows:

$$\text{precision}(L, M) = \frac{\sum_{t \in L, e \in t} enabled_L(e)}{\sum_{t \in L, e \in t} enabled_M(e)} \quad (6)$$

In summary, alignments are crucial to have accurate insights on the fitness and precision. However, as already acknowledged, they are hard to compute in general. In the remaining of this section, we briefly revise the challenge of computing alignments, together with some alternatives that have been proposed in recent years.

Computing Alignments. Computing an optimal alignment for an arbitrary combination of a process model and an event log is a far from trivial task. In terms of complexity, the task is as complex as reachability in Petri nets which, for general Petri nets, is undecidable. Nonetheless, several techniques exist to compute alignments. The best-known technique uses the A^* algorithm to find the shortest part in the reachability graph of the so-called synchronous product net [64]. This synchronous product is a combination of the process model and a Petri-net representation of the trace. Figure 7 shows an example of a synchronous product net for the running example. The algorithm associates costs to every transition in the synchronous product and uses these costs to find the shortest path from the initial marking to the final marking by expanding a minimal portion of the search space [67, 68].

When synchronous products become too large to handle for a monolithic algorithm decomposition approaches can be used [29] to decompose the construction of an alignment into smaller problems which can be combined into a full optimal alignment. If optimality is not a requirement, sub-optimal alignments can be identified with a variety of techniques [51, 52, 60, 62].

Another approach is the use of so-called satisfiability solvers [10]. The alignment problem is encoded as a SAT problem by translating the synchronous product to a set of boolean formulas. Because of this, the solution is limited

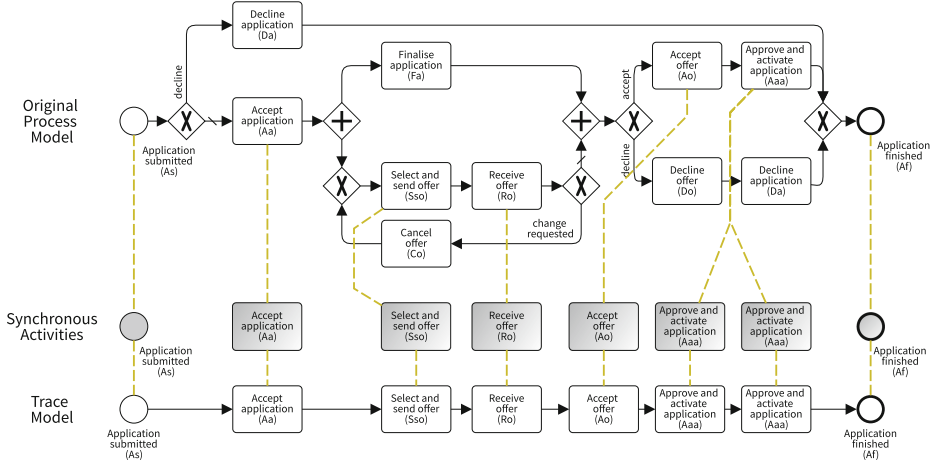


Fig. 7. The synchronous product model for the running example and trace $T_1 = \langle As, Aa, Sso, Ro, Ao, Aaa, Aaa \rangle$.

to safe Petri nets. While strictly a limitation, this is hardly a problem as most process modelling languages found in industry belong to this class of models. A third approach for computing alignments, which is bound by the same limitation, uses job-shop schedulers to find the optimal set of moves [19].

Finally, symbolic techniques exist to compute alignments [43]. These techniques have the upside that they can compute alignments for large sets of traces at once, rather than trace by trace as all techniques above do. However, the downside is that they rely on the state space of the process model to be known. In models with many parallel constructs, this state space may be prohibitively large. An approach using an implicit representation of the state space by means of a *Binary Decision Diagram* was presented recently which alleviates the aforementioned explosion [9].

3 Relating Observed and Modelled Behaviour: Advanced Techniques

In the previous sections, the focus of conformance checking was very much on control flow, i.e. the ordering of activities in the event log in relation to the specified order in which activities should be executed according to the process model. However, real-life processes are not only about activities. Instead, processes are executed by people within an organization to reach a certain business goal. This goal is expressed by data in the process and the process model serves as a guide to reach the goal as efficiently or as precisely as possible.

Consider, for example, the event log in Table 1. Next to the case identifier and the activity, we also see other data such as the amount of the application, the corresponding offer id sent to the customer and whether or not this offer is

signed. Not shown in this log is the identity of the employee who executed each activity, but it is not hard to imagine that companies have many employees of different roles and with different authorisation levels.

When doing conformance checking, it is important to consider all these elements and for this, more advanced conformance checking techniques, based on alignments, exist.

Data-Aware Alignments. Data plays a pivotal role in processes. Decisions are typically based on data that is provided at the start of a process or generated by any of the activities in the process. In our example of Table 1, the amount columns shows both types. Event e_{13} refers to an application being submitted by a customer, requesting a loan of 2000 euro. Event e_{37} subsequently shows that the bank offers the customer a loan for 1500 euro. In this process, the activity “Select and send offer” should not be executed with an amount higher than the requested amount. For application A5634 this is correct, but application A5636 shows a violation of this rule as the requested amount is only 200 euro, while the offered amount is 500.

To identify such data issues, several approaches exist. In [20,21] the authors first align the control flow using any of the techniques described above and then they check for deviations on the data level. This work is extended in [32] providing more control over the result and, especially, adopting a balanced view of control-flow and rules referring to the data perspective. Recently an approach that uses SMT solvers brings a fresh air to compute data-aware alignments [25].

Resource-Aware Alignments. Consider, for sake of argument, that the offered amount in our example can be higher than the requested amount, but only if the activity “select and send offer” is executed by a manager. In that case, the resource has a higher authorization level to actually deviate from the customer’s request. However, if this happens, the final activity “Approve and activate application” also needs to be executed by a manager and this should not be the same person (four-eyes principle).

The relation between the roles and resource identities across different activities makes checking this more complex than data-aware alignments. The authors of [3] consider the resource perspective by looking at the various data operations in an event log and checking if these operations are performed by authorized resources.

Integrated Approaches. The techniques presented above share a common feature that they first align the control flow and then use the control-flow alignment to check data and resource rules. An important downside of this approach is that certain deviations may not be detected. Consider, for example, a manager who decides to login to the bank’s system and read the application of his neighbour. As no activity is performed, the event log would not show any events and, when a data-access log is checked in isolation, the manager has the authority to read

application data, hence no data-access violation is found. However, the manager read data outside of the context of a process, i.e. there was no business-goal associated to the read action.

To comprehensively check the conformance of an event log from the viewpoint of the control flow, the data-access and resource authorization, a more recent approach has been developed by Mozafari et al. [5]. In their paper, the combination of an event log, a data access log and a resource model is used to construct a large synchronous product. This synchronous product is subsequently used to find optimal alignments with respect to deviations in all three perspectives combined without favouring one over the other. These deviations include, for example, spurious data access and authorization problems where otherwise authorized users access data outside of the context of the case they are working on.

Compliance Checking. The focus of conformance checking so far has been on the situation where end-to-end process models are available. However, in many companies, such process models do not exist. Instead, each process is only governed by a set of compliance rules, i.e. all activities can be performed, as long as these rules are not broken. Rule engines, as discussed earlier in the introduction, can typically raise flags when as soon as business rules are violated. However, a rule engine typically only recognizes the moment when a rule is violated. Conformance checking using alignments can also be used to identify that specific business rules are not yet fulfilled, but no violation occurred yet.

The work of Ramezani et al. [54] shows how typical compliance rules from the accountancy and control domain can be translated into small Petri nets which in turn can be aligned with event logs to identify violations against these rules as log- or model-moves.

Realtime (or Righttime) Conformance Checking. So far, conformance checking was discussed as a technique to identify deviations after processes have been concluded. However, in some cases, it may be interesting to detect deviations during the execution of a process [12,13,70]. Such techniques are often referred to as streaming techniques, i.e., data is being processed as it comes in and a realtime dashboard provides insights into the current conformance level of an entire process. This is particularly useful in environments where employees have a great deal of flexibility in executing activities within a process but where specific conditions have to be met at the end.

Conformance Checking Without Process Models. Finally, a specific type of conformance checking exists which does not rely on a traditional notion of a process model. Instead, the event log itself is used as a representative of both the correct and incorrect behaviour and deviations are detected between the mainstream behaviour prevalent in the event log and the ‘outlier’ cases [36,37]. Specifically, this approach employs recurrent neural networks (RNNs) that are

trained for next activity prediction moving through a trace forward or backward. These predictions can be seen as an approximation of a process model against which the alignments of traces are computed.

4 Applications of Conformance Checking

So far, we discussed essential techniques for conformance checking along with their generalization and extension to scenarios beyond the traditional, retrospective analysis of control-flow information. Next, we turn the focus to the broader field of applications of conformance checking.

We first note that an understanding of the link between the recorded and modelled behaviour of a process serves as a foundation for various model-based techniques for the analysis of qualitative and quantitative process properties. The importance of conformance checking for such analysis is detailed in Sect. 4.1, taking techniques for the analysis of performance characteristics and decision points as examples.

A second important observation relates to the fact that deviations between recorded and modelled behaviour, as revealed by conformance checking, can potentially be attributed to quality issues in the event log or the process model. Both, a log and a model, denote representations of the process at hand, which may be incomplete, outdated, imprecise, or simply wrong. This gives rise to a generalized notion of conformance checking, which aims at a separation of deviations that are due to quality issues in the event log or the process model. As discussed in Sect. 4.2, this generalized view on conformance checking enables us to describe common techniques for process mining as part of a unified framework.

4.1 The Case of Model-Based Process Analysis

Process models serve as the starting point for a plethora of process analysis techniques. Such analysis may be classified along various dimensions. That is, the point in time addressed by the analysis distinguishes retrospective, predictive, or even prescriptive analysis of a business process. The granularity of the analysis may be defined to be on individual instances of a process or a set thereof, thereby integrating potential interactions between different instances of a process. Moreover, analysis based on a process model may incorporate diverse process perspective, starting with the traditional view only on the control-flow of the process, through the data produced and consumed during its execution, the impact of such data on the control-flow, the integration of events produced by the environment in which the process is executed, the utilization of resources, and the definition of organizational responsibilities, to name just a few examples.

Regardless of the specific type of model-based process analysis, conformance checking provides a means to ensure that the models provide reasonable representation of the actual behaviour. Considering the behaviour as recorded in an event log as a representation of actual process execution, despite all potential issues related to data quality, such as accuracy and completeness of an event

log, conformance checking establishes trust into the analysis results obtained from the models. In the following paragraphs, we reflect on this application of conformance checking for three types of model-based analysis techniques.

Performance Analysis. Performance properties are an important aspect of process analysis in various domains. Here, specific measures include information on the time needed by a process instance from start to end, also known as cycle time or sojourn time, which is captured in terms of simple statistics, such as the average or maximal sojourn time, or complete distributions. Moreover, understanding how much time is needed to reach a certain milestone in the execution of a process is valuable information for operational process management, e.g., related to the scheduling of resources.

To enable the respective analysis, a process model is enriched with performance information. Common notions include simple annotations such as the average execution time per task. Yet, one may also consider more elaborated annotations, such as the distributions of not only the execution time per task, but also the wait time between the execution of subsequent tasks. Based on these annotations, analytical techniques or simulation are used to compute performance measures.

Given an event log, conformance checking that links the events of traces to the tasks in a process model helps to extract such performance annotations. For instance, once an alignment is computed, the synchronous steps indicate for which temporal information attached to events needs to be incorporated for the annotations for specific tasks. Note that this is particularly beneficial, once a model contains several tasks with the same label, i.e., representing the same activity of the process. In that case, an alignment separates the events that shall serve as the basis for the performance annotation of the different tasks based on the behavioural context in which they are executed. For instance, the model for a loan application process in Fig. 3 contains two tasks for declining an application (*Da*). This way, the respective activity may be executed in different contexts, once directly after the submission of the application and once towards the end of the process, after an offer has been declined. Consequence, both tasks may have different performance characteristics. Alignments help to incorporate these differences by separating the events that are linked to either task.

However, conformance checking may not only employed to extract performance annotations from an event log, but also enables their validation. For instance, performance annotations may have been defined manually, based on expectations. Then, temporal information of the event log may be utilized to validate these annotations, where, again, conformance checking indicates which events shall be considered for which of the tasks in the process model.

Decision Point Analysis. Decision point analysis aims at insights on the conditions that govern decision points in a process. In process modelling, it is a common abstraction to neglect such conditions and simply assume that a non-deterministic choice is taken, as the conditions may not be relevant for some

control-flow-oriented analysis. However, this abstraction may also be problematic, as it hides how the context of process execution influences the control-flow, e.g., that certain activities are executed solely for certain types of cases. Such insights are particularly relevant also for performance analysis as discussed above, since the conditions at a decision point may induce highly skewed distributions. In our running example, Fig. 3, there is a first decision point directly after the submission of an application, which may lead to an immediate rejection and, hence, completion of process execution. Understanding the properties of cases that govern this decision will, therefore, be very beneficial for any analysis of performance characteristics.

To understand the conditions at decision points of a process, decision mining may be employed. It takes traces of an event log, including the data attached to the events or the trace as a whole, as observations for particular decision outcomes. Then, a classification problem is derived, with the different outcomes being the classes, and common techniques for supervised classification enable the construction of a classifier. Assuming that the obtained classifier can be interpreted, e.g., is represented as a decision tree, the conditions for a decision point can be extracted and added to a process model.

In this context, conformance checking, again, helps to prepare a process model for analysis, as well as to validate existing annotations. In the former case, alignments that link events to tasks help to prepare the data needed for decision mining. Through an alignment, the data available at a specific decision point is characterised and may be used as input to the classification algorithm. The latter case, the decision points in a process model have already been annotated with the respective conditions. Then, conformance checking reveals if these conditions are matched with the behaviour recorded in the event log, either by constructing an alignment solely based on control-flow information and checking the conditions at decision points separately, or by integrating the conditions directly in the computation of multi-perspective alignments as discussed in Sect. 3.

4.2 A General View on Conformance Checking

An event log and a process model both denote representations of an abstract entity, the actual process as it is implemented in an organization. From this view point, illustrated in Fig. 8, it becomes clear that any deviation detected between these representations may potentially be attributed to the way that the representations capture the actual process, i.e., the log and the model may show quality issues. For instance, logging mechanisms may be faulty and the integration of event data from different systems may be imprecise. Similarly, models may have been created based on an incomplete understanding of the process and may be biased towards the expected rather than the actual behaviour. Moreover, in many application contexts, processes are subject to change and evolve over time. Hence, process models created at some time point become outdated. Event logs that span a large time period, in turn, may contain information about

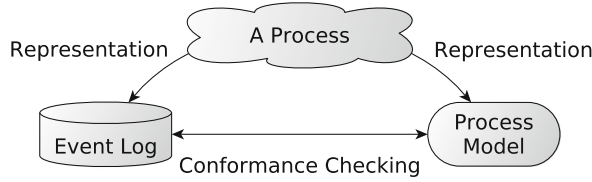


Fig. 8. Both, an event log and a process model, are representations of a process.

different versions of a process, so that the log in its entirety appears to describe a process that was actually never implemented as such at any specific point in time.

From the above observation, it follows that a deviation between an event log and a process model may be interpreted as an issue to fix in either of the representations. That is, one of the representations is assumed to be correct, i.e., it is assumed to truthfully denote the actual process, whereas the other representation is updated with the goal to resolve the deviation. Specifically, techniques to repair a process model based on the event log and techniques to repair an event log based on the process model have been proposed in the literature, as discussed next.

Model Repair. Assuming that an event log constitutes a correct representation of a process' behaviour, deviations detected between the log and a process model are a starting point for model repair. To this end, existing techniques are mostly based on alignments computed between a trace of an event log and the process model. The reason being that alignments clearly separate behaviour that is only observed in the process model (i.e., a move in model) and behaviour that is present only in the trace (i.e., a move in log).

Intuitively, a move in model captures the situation that the execution of an activity is defined to be mandatory, while this execution is optional according to the supposedly correct event log. Therefore, a simple repair strategy is to relax the control-flow defined by the model and explicitly enable the continuation of a process instance without executing the respective activity. Note though that different syntactical changes may be considered to realize this change. For instance, in a BPMN process model, one may insert a decision point before the task to determine whether it is executed, whereas a similar effect may also be achieved by changing the semantics of existing routing constructs, such as transforming a parallel split into an inclusive choice.

A move in log, on the other hand, hints at a supposedly correct activity execution that is without counterpart in the model. A repair strategy, therefore, is to insert a corresponding task into the process model. The location for this insertion is also determined based on the conformance checking result. That is, the alignment up to the respective move in log induces a state in the process

model. The task needs to be inserted, such that it is activated in this state and such that before and after its execution, all tasks that have been activated in the original state are still activated. In practice, such a repair operation may not only be conducted on the level of individual model in log steps, but for sequences thereof. In this case, a model fragment to capture the behaviour of this sequence is discovered and inserted into the original model.

As an example, consider the following alignment for the process model introduced earlier (Fig. 3).

log trace	<i>As</i>	<i>Aa</i>	<i>Sso</i>	<i>Ro</i>	$\gg$	<i>Ao</i>	<i>Aaa</i>	<i>Aaa</i>	$\gg$	
execution sequence	<i>As</i>	<i>Aa</i>	<i>Sso</i>	<i>Ro</i>	<i>Fa</i>	<i>Ao</i>	<i>Aaa</i>	$\gg$	<i>Af</i>	

From the move in model ($\gg, Fa$), one may derive a change in the process model that enables skipping of the respective task *Fa* in the process model. The move in log (*Aaa*, $\gg$), in turn, suggests a change in the model that supports an additional execution of the activity to approve and activate an application. Yet, we note that this activity execution directly succeeds a synchronous move for a task referring to same activity. Hence, instead of adding a new task in the process model, it may be more desirable to generalise the process model and insert a loop around the existing task *Aaa*, so that it may be executed multiple times in an execution sequence of the model.

Log Repair. The idea of repairing a process representation based on the results of conformance checking may also be applied to event logs. Given an alignment of a trace and a process model, the actual changes to apply to the trace are derived from the types of the respective alignment steps. Under the assumption that the model is a correct representation of the process, a move in model would lead to the insertion of an event into the trace at the position of the alignment step. An event that is part of a move in log, in turn, would be deleted from the trace.

In practice, the insertion or deletion of events of a trace may be problematic. For instance, the creation of artificial events raises the question of how to define the values of an events' attributes, from generic ones such as an events' timestamp to domain-specific attributes (e.g., the state of a business object). Against this background, log repair may not focus on alignment steps in isolation, but aim at identifying high-level changes. An example would be the presence of two alignment steps, a move in model and a move in log, both related to the execution of the same activity. Instead of deleting and inserting an event, moving the event from the position of the move in log step to the position of the move in model step would enable repair without the need to generate an artificial event.

Taking up the aforementioned example, based on the alignment, log repair may suggest that the second event linked to the approval and activation of the application (*Aaa*) is erroneous (e.g., the activity execution was recorded twice due to a faulty logging mechanism) and, thus, shall be removed from the trace.

At the same time, it may suggest to insert events for the activities to finalise the application (Fa) and finish the application (Af), for instance, assuming that these steps are manually recorded, so that some incompleteness of the trace is to be expected.

Generalized Conformance Checking. Both, model repair and log repair consider one process representation to be correct, which may therefore serve as a ground truth. In the general case, however, quality issue may be present in both representations. As a consequence, some of the conformance issues detected between a model and a log may stem from the model not adequately capturing the process, some of them may originate from low quality of the event log, while some are also inherent deviations that need to be analysed.

To balance the different reasons of conformance issues, it was suggested to incorporate a notion of *trust* in the process model, denoted by $\tau_M \in [0, 1]$, as well as the event log, denoted by $\tau_L \in [0, 1]$ [48]. These trust values capture the assumed correctness of either representation and may reflect how the representation has been derived. For instance, a process model created as part of a first brainstorming session may be less trustworthy in terms of correctness and completeness compared to a model created as a part of a rigorous process management initiative. Similarly, an event log created by a process-oriented information systems can, in general, be expected to be more trustworthy than a manual documentation of activity executions by a diverse group of process stakeholders.

Once a trust level has been specified for both, the model and the log, conformance checking may be phrased as an optimization problem that incorporates model and log repair. To this end, the following notions need to be defined: A function δ_{L^2} to measure the distance of two event logs; a function δ_{M^2} to measure the distance of two models; and a function $\delta_{L,M}$ to measure the distance of an event log and a process model, such as alignment-based fitness or a combination of fitness and precision. Given an event log L and a process model M , generalized conformance checking [48] is then defined as the identification of some adapted log L^* and adapted model M^* , such that:

$$L^*, M^* = \operatorname{argmin}_{L', M'} (\delta_{L^2}(L, L'), \delta_{M^2}(M, M'), \delta_{L,M}(L', M'))$$

$$\text{subject to } \delta_{L^2}(L, L') \leq 1 - \tau_L \text{ and } \delta_{M^2}(M, M') \leq 1 - \tau_M.$$

Intuitively, the above problem formulation considers that the given model M and log L may require to be adapted, if they are not fully trustworthy. However, the trust values induce a bound for the distance between any adapted model and log, and the original model and log, respectively, as illustrated in Fig. 9. Within the space set by these bounds, the distances between the adapted and original model, between the adapted and original logs, and between the adapted model and the adapted log shall be minimised. Here, a specific instantiation may require the minimisation of a linear combination of the three distances.

Generalized conformance checking unifies various tasks in the field of process mining [14, 48]. Table 4 highlights how specific tasks can be seen as instances

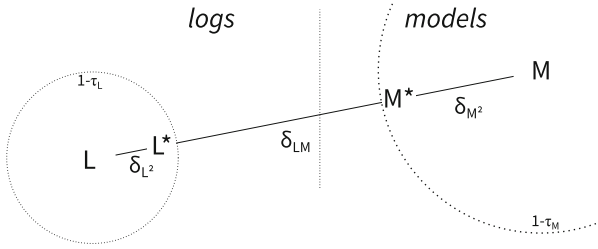


Fig. 9. The problem of generalized conformance checking (from [14]).

Table 4. Overview of process mining tasks listed as instances of the generalised conformance checking problem according to [14, 48].

Process mining task	Log Trust	Model Trust
Classical Process Discovery finds a model that best fits to the entire event log, e.g., the alpha algorithm [65]	$\tau_L = 1$	$\tau_M = 0$
Heuristic Process Discovery algorithms apply preprocessing to the event log by discarding infrequent patterns [26, 79]	$0 < \tau_L < 1$	$\tau_M = 0$
Model Repair fixes deficient models due to, e.g., a change in the system that is reflected in the log. For example [24]	$\tau_L = 1$	$0 < \tau_M < 1$
Conformance Checking tries to find misalignments between event log and model. Example works include [50, 53, 64]	$\tau_L = 1$	$\tau_M = 1$
Log Repair modifies the log such that it better conforms to the given trusted model [47, 74]	$0 < \tau_L < 1$	$\tau_M = 1$
“Happy Path” Simulation is complementary to heuristic process discovery. It is a theoretical use case where we do not trust infrequent parts of the model [33]	$\tau_L = 0$	$0 < \tau_M < 1$
Process Simulation is complementary to process discovery, where we are given an untrustworthy empty log and a fully trustworthy model	$\tau_L = 0$	$\tau_M = 1$
Garbage In, Garbage Out. When both the model and the log are untrustworthy, the best log and model tuple that fits them is any pair of model and log that fits each other, including an empty log and an empty model	$\tau_L = 0$	$\tau_M = 0$
Generalised Conformance Checking answers the question where the model would best be adopted, and where the log would best be adopted for a better overall fit. This goes beyond alignments, as the latter only detect the misalignments without specifying which side is to “blame”	$0 < \tau_L < 1$	$0 < \tau_M < 1$

of the generalized conformance checking problem, depending on the trust into an event log or a process model. Specifically, tasks such as classical process discovery or process simulation fit into this picture when assuming that there is no trust into the model or the log, which can be interpreted as the setting that the respective artifacts are not available.

5 Further Reading

Conformance checking has evolved significantly in the last decade, enabling the industrial adoption and commercial software offerings. As it has been shown in Sect. 3, techniques beyond control flow are already been proposed in the last years, since considering other perspectives brings significant value and triggers adoption.

Still, the core of the techniques developed are still focusing on the algorithmic aspects of the computation of conformance artefacts for the control flow perspective. We now review further work on the three dimensions considered in this chapter, thereby providing pointers for further reading.

Rule Checking. The idea of rule-based conformance checking is to rely on constraints which are then checked for the traces of an event log. The idea of rule-based conformance checking has been brought forward in [76]. It employs constraints derived from the (causal) behavioural profile of a process model [75, 77], which are sets of binary relations over activities derived from the order of potential occurrences of tasks in the execution sequences of the model.

This general idea, however, is not limited to a specific set of rules. Rather, other notions of constraints can be used in the very same manner, including transition adjacency relations [80] and the rules of the 4C spectrum [42]. Such sets of binary rules to capture behaviour are inherently limited in their expressiveness, though, as already for relatively simple classes of models, an exponentially growing number of rules would be needed to capture the complete behaviour [40]. Rule-based conformance checking, therefore, lends itself to scenarios, where only certain constraints need to be checked rather than the complete behaviour as specified by a process model.

While the results of rule checking enable insights on deviant traces, they may also be used for aggregated conformance measures. For instance, fitness measures may be derived based on the numbers of satisfied and violated rules [76]. Also, filtering of rule violations and discovery of associations between them may provide further insights into context of non-conformance [76, 78].

Finally, conformance checking based on rules has the advantage that it can be lifted to online scenarios in a straight-forward manner. To this end, rules can be translated to queries over streams of events, see [17, 78], which enables the use of algorithms and systems developed for complex event processing [16].

Token Replay. Techniques for token replay were first introduced in [49]. Alternative techniques were presented in [72], and later adapted to an online scenario in [71]. Recently new heuristics have been recently proposed that make token replay a fast alternative to alignments [7, 8].

Alignments. The seminal work in [1] proposed the notion of alignment and developed a technique based on A* to compute optimal alignments for a particular class of process models. Improvements of this approach have been presented

recently in different papers [66,68]. The approach represents the state-of-the-art technique for computing alignments, and can be adapted (at the expense of increasing significantly the memory footprint) to provide all optimal alignments. Alternatives to A* have appeared in the last years: in the approach presented in [19], the alignment problem is mapped as an *automated planning* instance. Automata-based techniques have also appeared [31,44]. The techniques in [44] (recently extended in [45]) rely on state-space exploration and determination of the automata corresponding to both the event log and the process model, whilst the technique in [31] is based on computing several subsets of activities and projecting the alignment instances accordingly. We also highlight the recent approach that is grounded on the use of relaxation labelling combined with A*, to provide a light alternative to compute alignments [39].

The work in [57] presented the notion of *approximate* alignment to alleviate the computational demands by proposing a recursive paradigm on the basis of the structural theory of Petri nets. In spite of resource efficiency, the solution is not guaranteed to be executable. Alternatively, the technique in [59] presents a framework to reduce a process model and the event log accordingly, with the goal of alleviating the computation of alignments. The obtained alignment, called *macro-alignment* since some of the positions are high-level elements, is expanded based on the information gathered during the initial reduction. Techniques using local search have recently been also proposed very recently [61].

Against this background, the process mining community has focused on divide-and-conquering the problem of computing alignments, as a valid alternative to this problem with the aim of alleviating its complexity without degrading the quality of the solutions found. We turn now our focus to decompositional approaches to compute alignments, which are more related to the research of this paper.

Decompositional techniques have been presented [35,63,73] that, instead of computing optimal alignments, they focus on the *crucial problem* of whether a given trace fits or not a process model. These techniques vertically decompose the process model into pieces satisfying certain conditions (so only *valid* decompositions [63], which satisfy restrictive conditions on the labels and connections forming a decomposition, guarantee the derivation of a real alignment). Later on, the notion of *recomposition* has been proposed on top of decompositional techniques, in order to obtain optimal alignments whenever possible by iterating the decompositional methods when the required conditions do not hold [29]. In contrast to the aforementioned vertical decomposition techniques, this approach does not require this last consolidation step of partial solutions, and therefore can be a fast alternative to these methods at the expense of losing the guarantee of optimality.

There has been related work also on the use of partial order representations of process models for computing alignments. In [13], unfoldings were used to capture all possible transition relations of a model so that they can be used for online conformance checking. In contrast, unfoldings were used recently in a series of papers [38,60] to speed-up significantly the computation of alignments.

We believe these approaches, specially the last two, can be easily integrated in our framework.

Also, the work of [45] can also be considered a decompositional approach, since it proposes decomposing the model into sequential elements (*S-components*) so that the state-space explosion of having concurrent activities is significantly alleviated. We believe that this work is quite compatible with the framework suggested in this paper, since the model restrictions assumed in [45] are satisfied by the partial models arising from our horizontal decomposition.

Finally, the MapReduce distributed programming model has already been considered for process mining. For instance, Evermann applies it to process discovery [23], whilst [15] applies it for monitoring declarative business processes. Recently, MapReduce techniques has been proposed to offer a horizontal decompositional alternative to computing alignments [62].

6 Milestones and Challenges

Conformance checking is nowadays a mature field, demonstrated by its presence in some of the process mining commercial tools and process mining use cases. In spite of this, the available support for its adoption is far from complete. One example is the metrics available: whilst fitness or precision are considered well evaluated through current techniques, accurate generalization metrics that additionally can be evaluated efficiently are yet to come [41, 69, 72].

Alignments are a central pillar of current techniques for conformance checking. However, the complexity requirements of the state-of-the-art techniques hamper their application for large instances (see Sect. 2.4). Actually, process mining is facing the following paradox: whilst there exist techniques to discover process models arbitrarily large [4, 30], most of the existing alignment computation techniques will not be able to handle such models. Alternative approaches, like the decomposition or structural techniques only alleviate the problem, at the expense of losing the guarantee of important properties like optimality. Also, when incorporating other dimensions like data or resources, so that a multi-perspective for conformance checking is enabled, the complexity of the problem increases significantly, making it difficult to be applied for real-life problems; we envision new contributions also for multi-perspective conformance checking in the near future that can overcome this limitation.

Beyond computational or algorithmic challenges, there are other equally important challenges, more oriented towards considering the understanding of conformance checking results. One of them is the visualization of deviations. In industrial scenarios, thousand of deviations can easily pop up when assessing conformance, and it is not so easy to rank the importance of each one with a criteria that really impacts the business of the organization. For instance, looking at Fig. 1, one can see the list of violations at the bottom of the figure, ordered by the percentage of the cases where these deviations occur. This may not necessarily be the most interesting ranking from a business perspective.

We now provide a list of particular challenges with the aim of triggering future research in the field. The list is by no means complete.

Representing Uncertainty and Preventing Bias. As mentioned above, conformance checking deals with the comparison of recorded behaviour against specified behaviour, typically represented as an event log and process model. Based on this comparison, conclusions can be drawn with respect to the recorded behaviour as well as the underlying process which produced the recorded behaviour.

This distinction becomes only irrelevant when the recorded behaviour contains all the process behaviour of interest. In all other situations, where the observed behaviour is only a sample of the complete process behaviour, a source of variation is introduced by the sample. Sampling variation will cause the outcome of the conformance checking activity, which is only an estimate of the true value, to vary over different samples. Initial work on this direction has been recently proposed [6,28].

Information on the accuracy of a specific conformance estimate is important for a practitioner to make informed decisions. Unfortunately, representing uncertainty is typically ignored by existing conformance checking techniques and remains an important open challenge.

A second related challenge is that practitioners not only want an idea about the estimate's accuracy, but also want some guarantee that the estimate is unbiased. Various sources of errors exist which could lead to biased results, which receive too little attention in the existing work on conformance checking.

Some of the most relevant sources of errors are coverage error and construct validity. Coverage error occurs when the recorded behaviour is not a representative sample of the underlying process under analysis. This could be caused e.g. by non-random sampling or an incorrect definition of the underlying process. Construct validity refers to the question whether the conformance technique actually measures what it claims to be measuring.

This issue of estimate bias raises at least three important challenges. Firstly, there is a need for further development of conformance techniques which produce unbiased estimates, as recent research empirically challenged the claim that existing estimators are unbiased [27]. Secondly, with respect to construct validity, more attention should be given to making explicit what a measure actually represents. In particular the concept of generalisation suffers from an ambiguous and unclear definition, while other conformance measures are so complex that it is no longer clear what is measured and how it behaves. Thirdly, illustrating that a conformance estimator is unbiased should become a fundamental methodological part of any paper introducing and reviewing conformance checking techniques.

Computational Feasibility. As with many data analysis tasks, computation feasibility is a challenge. In the context of conformance checking, different elements contribute to this. One element lies in the current approach itself. As we highlighted before, alignment-based approaches are the state-of-the-art techniques to conformance checking due to its robustness and detailed diagnosis on deviations at the event level. However, it is also a computationally intensive

operation that can take a long time to execute and can even be unfeasible for industrial-sized processes.

Further, computational feasibility is challenged by the persistently growing size of event logs. In the industry, huge quantities of events are recorded. For example, Boeing jet engines can produce ten terabytes of operational information every thirty minutes and Walmart is logging one million customer transactions per hour. In these contexts, operational efficiency is typically of paramount importance and is ensured by having predefined operational protocols and guidelines. Consequently, aside from being capable of dealing with complex and large underlying processes, conformance checking techniques should also support large amounts of data.

Responding to these computational challenges, techniques that are tailored for the emergence of large event logs and processes are created. For example, it is often not possible to store all the event data produced by large processes due to the limitation of storage capacity. This has motivated techniques that allow conformance checking to be performed in an online setting to data streams that are continuously producing event data related to ongoing cases. While a solution for one challenge, this response in itself holds additional challenges.

Online Conformance Checking. Online conformance checking analyzes event streams to assess their conformance with respect to a given reference model (the reader is referred to [11]) The key aspect of this problem is that events must be analyzed immediately after they are generated (without storing them). The key benefit of this technique is to be able to detect deviations immediately, thus giving time to the process manager to shift the trace back to the reference behaviour. More generally, the main benefit is the reduction in latency among the BPM lifecycle phases.

Event streams represent a specific type of data streams, where each data point is an event (as in standard event logs). General data stream mining techniques have been studied in the past and several stream operations models have been defined, including: insert-only streams; insert-delete streams; and additive streams. Respectively, events are only inserted; deleted; or “incremented” (this holds typically for numerical variables). Typically, event streams are assumed to be insert-only streams, where events are just added to the stream.

Since event streams are generally assumed to be unbounded and events are supposed to arrive at unpredictable pace, several constraints are imposed on the analysis. Specifically, once an element is received, it should be processed immediately: either it is analyzed or it is discarded. In case it is analyzed, it cannot be explicitly inspected again at a later stage: since the stream is unbounded, it is impossible to store it and its events have to be stored in an aggregated (or summarized) fashion. Additionally, the time scale plays a fundamental role in online conformance checking: a recent deviation is more important than older ones, as the process manager can immediately enact proper countermeasures.

Several problems are still open, for example how to find good conformance measures, which operate in efficient (i.e., constant) time. Other relevant and

unsolved problems are handling streams where the arrival time of events does not coincide with the execution time (thus, events need to be “sorted” afterwards), or understanding when a process instance is really terminated (even if the termination state of the model has been reached).

Desired Properties of Conformance Measures. The objective of conformance checking is to provide insights on how well a model describes given event data or how well given event data describes the model. This is represented both quantitatively - for measuring conformance - and qualitatively - for providing diagnostic information. We discuss properties and challenges of measures and diagnostics information in conformance checking.

Similar to machine learning, conformance measures are used to assess how well a model describes the event data: A model should have a high fitness or recall to the log (be able to replay all observed traces) and a high precision to the log (show little additional behaviour). Models with high fitness and precision distinguish themselves further in terms of generalization (their ability to replay likely, but so far not observed traces of the process that generates the log) and simplicity (being structurally simple). In this sense, we use conformance measures to compare two different models M1 and M2 with each other in their ability to describe a given log L (in terms of fitness and precision), describe the unknown process P behind the log (in terms of generalization), and be easily understandable (through a simple model structure). A model M1 scoring higher than a model M2 in a measure is considered to be the “better” model. For most event logs, the quality measures define a pareto front: a model scoring better on one measure scores worse on another measure leading to a set of “best” models for which no model can be found scoring better on any measure without scoring worse on other measures. With these properties, conformance measures have two main applications: helping a user decide which among a set of possible models is a preferred description of the event data, evaluating and benchmarking algorithms in process mining.

As it has been recently suggested [55], establishing certain axioms is a safe way to be able to determine the boundaries of a certain technique for determining a particular quality dimension. These axioms are expected to clarify important aspects such as logical consistency, robustness, confidence, to name a few.

7 Conclusions

This chapter provided an overview on conformance checking, aiming at covering the basic techniques, pinpointing what are the natural applications of the field and looking into the future by listing challenges that we believe will be crucial to overcome in the years to come. The chapter may be seen as a gentle introduction to the reference book in the field, where most of the topics are extensively developed [14].

Acknowledgements. This work has been supported by MCIN/AEI funds under grant PID2020-112581GB-C21.

References

1. Adriansyah, A.: Aligning observed and modeled behavior. Ph.D. thesis, Technische Universiteit Eindhoven (2014)
2. Adriansyah, A., Munoz-Gama, J., Carmona, J., van Dongen, B.F., van der Aalst, W.M.P.: Measuring precision of modeled behavior. *Inf. Syst. E-Bus. Manag.* **13**(1), 37–67 (2015)
3. Alizadeh, M., Lu, X., Fahland, D., Zannone, N., van der Aalst, W.M.P.: Linking data and process perspectives for conformance analysis. *Comput. Secur.* **73**, 172–193 (2018)
4. Augusto, A., Conforti, R., Dumas, M., La Rosa, M., Polyvyanyy, A.: Split miner: automated discovery of accurate and simple business process models from event logs. *Knowl. Inf. Syst.* **59**(2), 251–284 (2018). <https://doi.org/10.1007/s10115-018-1214-x>
5. Mozafari Mehr, A.S., de Carvalho, R.M., van Dongen, B.: Detecting privacy, data and control-flow deviations in business processes. In: Nurcan, S., Korthaus, A. (eds.) CAiSE 2021. LNBIP, vol. 424, pp. 82–91. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-79108-7_10
6. Bauer, M., van der Aa, H., Weidlich, M.: Sampling and approximation techniques for efficient process conformance checking. *Inf. Syst.* **104**, 101666 (2022)
7. Berti, A., van der Aalst, W.M.P.: Reviving token-based replay: Increasing speed while improving diagnostics. In: van der Aalst, W.M.P., Bergenthum, R., Carmona, J. (eds.) Proceedings of the International Workshop on Algorithms & Theories for the Analysis of Event Data 2019 Satellite Event of the Conferences: 40th International Conference on Application and Theory of Petri Nets and Concurrency Petri Nets 2019 and 19th International Conference on Application of Concurrency to System Design ACSD 2019, ATAED@Petri Nets/ACSD 2019, Aachen, Germany, 25 June 2019, vol. 2371 of CEUR Workshop Proceedings, pp. 87–103. CEUR-WS.org (2019)
8. Berti, A., van der Aalst, W.M.P.: A novel token-based replay technique to speed up conformance checking and process enhancement. *Trans. Petri Nets Other Model. Concurr.* **15**, 1–26 (2021)
9. Bloemen, V., van de Pol, J., van der Aalst, W.M.P.: Symbolically aligning observed and modelled behaviour. In: 18th International Conference on Application of Concurrency to System Design, ACSD 2018, Bratislava, Slovakia, 25–29 June 2018, pp. 50–59 (2018)
10. Boltenhagen, M., Chatain, T., Carmona, J.: Optimized SAT encoding of conformance checking artefacts. *Computing* **103**(1), 29–50 (2020). <https://doi.org/10.1007/s00607-020-00831-8>
11. Burattin, A.: Streaming process mining. In: van der Aalst, W.M.P., Carmona, J. (eds.) Process Mining Handbook. LNBIP, vol. 448, pp. xx–yy. Springer, Cham (2022)
12. Burattin, A., Carmona, J.: A framework for online conformance checking. In: Teniente, E., Weidlich, M. (eds.) BPM 2017. LNBIP, vol. 308, pp. 165–177. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-74030-0_12

13. Burattin, A., van Zelst, S.J., Armas-Cervantes, A., van Dongen, B.F., Carmona, J.: Online conformance checking using behavioural patterns. In: Weske, M., Montali, M., Weber, I., vom Brocke, J. (eds.) BPM 2018. LNCS, vol. 11080, pp. 250–267. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-98648-7_15
14. Carmona, J., van Dongen, B.F., Solti, A., Weidlich, M.: Conformance Checking - Relating Processes and Models. Springer, Cham (2018). <https://doi.org/10.1007/978-3-319-99414-7>
15. Chesani, F., Ciampolini, A., Loreti, D., Mello, P.: Map reduce autoscaling over the cloud with process mining monitoring. In: Helfert, M., Ferguson, D., Méndez Muñoz, V., Cardoso, J. (eds.) CLOSER 2016. CCIS, vol. 740, pp. 109–130. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-62594-2_6
16. Cugola, G., Margara, A.: Processing flows of information: from data stream to complex event processing. ACM Comput. Surv. **44**(3), 15:1–15:62 (2012)
17. Daum, M., Götz, M., Domaschka, J.: Integrating CEP and BPM: how CEP realizes functional requirements of BPM applications (industry article). In: Bry, F., Paschke, A., Eugster, P.Th., Fetzer, C., Behrend, A. (eds.) Proceedings of the Sixth ACM International Conference on Distributed Event-Based Systems, DEBS 2012, Berlin, Germany, 16–20 July 2012, pp. 157–166. ACM (2012)
18. de Leoni, M.: Foundations of process enhancement. In: van der Aalst, W.M.P., Carmona, J. (eds.) Process Mining Handbook. LNBIP, vol. 448, pp. xx–yy. Springer, Cham (2022)
19. de Leoni, M., Marrella, A.: Aligning real process executions and prescriptive process models through automated planning. Expert Syst. Appl. **82**, 162–183 (2017)
20. de Leoni, M., van der Aalst, W.M.P.: Aligning Event logs and process models for multi-perspective conformance checking: an approach based on integer linear programming. In: Daniel, F., Wang, J., Weber, B. (eds.) BPM 2013. LNCS, vol. 8094, pp. 113–129. Springer, Heidelberg (2013). https://doi.org/10.1007/978-3-642-40176-3_10
21. de Leoni, M., van der Aalst, W.M.P., van Dongen, B.F.: Data- and resource-aware conformance checking of business processes. In: Abramowicz, W., Kriksciuniene, D., Sakalauskas, V. (eds.) BIS 2012. LNBIP, vol. 117, pp. 48–59. Springer, Heidelberg (2012). https://doi.org/10.1007/978-3-642-30359-3_5
22. Dumas, M., van der Aalst, W.M.P., ter Hofstede, A.H.M.: Process-Aware Information Systems: Bridging People and Software Through Process Technology. Wiley, Hoboken (2005)
23. Evermann, J.: Scalable process discovery using map-reduce. IEEE Trans. Serv. Comput. **9**(3), 469–481 (2016)
24. Fahland, D., van der Aalst, W.M.P.: Model repair - aligning process models to reality. Inf. Syst. **47**, 220–243 (2015)
25. Felli, P., Gianola, A., Montali, M., Rivkin, A., Winkler, S.: CoCoMoT: conformance checking of multi-perspective processes via SMT. In: Polyvyanyy, A., Wynn, M.T., Van Looy, A., Reichert, M. (eds.) BPM 2021. LNCS, vol. 12875, pp. 217–234. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-85469-0_15
26. Günther, C.W., van der Aalst, W.M.P.: Fuzzy mining – adaptive process simplification based on multi-perspective metrics. In: Alonso, G., Dadam, P., Rosemann, M. (eds.) BPM 2007. LNCS, vol. 4714, pp. 328–343. Springer, Heidelberg (2007). https://doi.org/10.1007/978-3-540-75183-0_24
27. Janssenswillen, G., Depaire, B.: Towards confirmatory process discovery: making assertions about the underlying system. Bus. Inf. Syst. Eng. **61**, 1–16 (2019)

28. Kabierski, M., Lam Nguyen, H., Grunske, L., Weidlich, M.: Sampling what matters: relevance-guided sampling of event logs. In: Di Ciccio, C., Di Francescomarino, C., Soffer, P. (eds.) 3rd International Conference on Process Mining, ICPM 2021, Eindhoven, Netherlands, 31 - November 4, 2021, pp. 64–71. IEEE (2021)
29. Lee, W.L.J., Verbeek, H.M.W., Munoz-Gama, J., van der Aalst, W.M.P., Sepúlveda, M.: Recomposing conformance: closing the circle on decomposed alignment-based conformance checking in process mining. *Inf. Sci.* **466**, 55–91 (2018)
30. Leemans, S.: Robust process mining with guarantees. Ph.D. thesis, Eindhoven University of Technology (2017)
31. Leemans, S.J.J., Fahland, D., van der Aalst, W.M.P.: Scalable process discovery and conformance checking. *Softw. Syst. Model.* **17**(2), 599–631 (2018)
32. Mannhardt, F., de Leoni, M., Reijers, H.A., van der Aalst, W.M.P.: Balanced multi-perspective checking of process conformance. *Computing*, **98**(4), 407–437 (2016)
33. Marquard, M., Shahzad, M., Slaats, T.: Web-based modelling and collaborative simulation of declarative processes. In: Motahari-Nezhad, H.R., Recker, J., Weidlich, M. (eds.) BPM 2015. LNCS, vol. 9253, pp. 209–225. Springer, Cham (2015). https://doi.org/10.1007/978-3-319-23063-4_15
34. Muñoz-Gama, J., Carmona, J.: A fresh look at precision in process conformance. In: Hull, R., Mendling, J., Tai, S. (eds.) BPM 2010. LNCS, vol. 6336, pp. 211–226. Springer, Heidelberg (2010). https://doi.org/10.1007/978-3-642-15618-2_16
35. Munoz-Gama, J., Carmona, J., van der Aalst, W.M.P.: Single-entry single-exit decomposed conformance checking. *Inf. Syst.* **46**, 102–122 (2014)
36. Nolle, T.: Process learning for autonomous process anomaly correction. Ph.D. thesis, Technical University of Darmstadt, Germany (2020)
37. Nolle, T., Seeliger, A., Thoma, N., Mühlhäuser, M.: DeepAlign: alignment-based process anomaly correction using recurrent neural networks. In: Dustdar, S., Yu, E., Salinesi, C., Rieu, D., Pant, V. (eds.) CAiSE 2020. LNCS, vol. 12127, pp. 319–333. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-49435-3_20
38. Padró, L., Carmona, J.: Approximate computation of alignments of business processes through relaxation labelling. In: Hildebrandt, T., van Dongen, B.F., Röglinger, M., Mendling, J. (eds.) BPM 2019. LNCS, vol. 11675, pp. 250–267. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-26619-6_17
39. Padró, L., Carmona, J.: Computation of alignments of business processes through relaxation labeling and local optimal search. *Inf. Syst.* **104**, 101703 (2022)
40. Polyvyanyy, A., Armas-Cervantes, A., Dumas, M., Garcia-Banuelos, L.: On the expressive power of behavioral profiles. *Formal Aspects Comput.* **28**(4), 597–613 (2016). <https://doi.org/10.1007/s00165-016-0372-4>
41. Polyvyanyy, A., Moffat, A., Garcia-Banuelos, L.: Bootstrapping generalization of process models discovered from event data (2021)
42. Polyvyanyy, A., Weidlich, M., Conforti, R., La Rosa, M., ter Hofstede, A.H.M.: The 4C spectrum of fundamental behavioral relations for concurrent systems. In: Ciardo, G., Kindler, E. (eds.) PETRI NETS 2014. LNCS, vol. 8489, pp. 210–232. Springer, Cham (2014). https://doi.org/10.1007/978-3-319-07734-5_12
43. Reißner, D., Armas-Cervantes, A., Conforti, R., Dumas, M., Fahland, D., La Rosa, M.: Scalable alignment of process models and event logs: an approach based on automata and s-components. *Inf. Syst.* **94**, 101561 (2020)
44. Reißner, D., Conforti, R., Dumas, M., La Rosa, M., Armas-Cervantes, A.: Scalable conformance checking of business processes. In: OTM CoopIS, Rhodes, pp. 607–627 (2017)

45. Reißner, D., Armas-Cervantes, A., Conforti, R., Dumas, M., Fahland, D., La Rosa, M.: Scalable alignment of process models and event logs: an approach based on automata and s-components. *Inf. Syst.* **94**, 101561 (2020)
46. Roehm, H., Oehlerking, J., Woehrle, M., Althoff, M.: Model conformance for cyber-physical systems: a survey. *Trans. Cyber Phys. Syst.* **3**(3), 30:1–30:26 (2019)
47. Rogge-Solti, A., Mans, R.S., van der Aalst, W.M.P., Weske, M.: Improving documentation by repairing event logs. In: Grabis, J., Kirikova, M., Zdravkovic, J., Stirna, J. (eds.) *PoEM 2013. LNBIP*, vol. 165, pp. 129–144. Springer, Heidelberg (2013). https://doi.org/10.1007/978-3-642-41641-5_10
48. Rogge-Solti, A., Senderovich, A., Weidlich, M., Mendling, J., Gal, A.: In log and model we trust? A generalized conformance checking framework. In: La Rosa, M., Loos, P., Pastor, O. (eds.) *BPM 2016. LNCS*, vol. 9850, pp. 179–196. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-45348-4_11
49. Rozinat, A.: Process mining conformance and extension. Ph.D. thesis, Technische Universiteit Eindhoven (2010)
50. Rozinat, A., van der Aalst, W.M.P.: Conformance checking of processes based on monitoring real behavior. *Inf. Syst.* **33**(1), 64–95 (2008)
51. Fani Sani, M., Garza Gonzalez, J.J., van Zelst, S.J., van der Aalst, W.M.P.: Conformance checking approximation using simulation. In: van Dongen, B.F., Montali, M., Wynn, M.T. (eds.) *2nd International Conference on Process Mining, ICPM 2020, Padua, Italy, 4–9 October 2020*, pp. 105–112. IEEE (2020)
52. Fani Sani, M., van Zelst, S.J., van der Aalst, W.M.P.: Conformance checking approximation using subset selection and edit distance. In: Dustdar, S., Yu, E., Salinesi, C., Rieu, D., Pant, V. (eds.) *CAiSE 2020. LNCS*, vol. 12127, pp. 234–251. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-49435-3_15
53. Senderovich, A., Weidlich, M., Yedidsion, L., Gal, A., Mandelbaum, A., Kadish, S., Bunnell, C.A.: Conformance checking and performance improvement in scheduled processes: a queueing-network perspective. *Inf. Syst.* **62**, 185–206 (2016)
54. Taghiabadi, E.R., Gromov, V., Fahland, D., van der Aalst, W.M.P.: Compliance checking of data-aware and resource-aware compliance requirements. In: Meersman, R., Panetto, H., Dillon, T., Missikoff, M., Liu, L., Pastor, O., Cuzzocrea, A., Sellis, T. (eds.) *OTM 2014. LNCS*, vol. 8841, pp. 237–257. Springer, Heidelberg (2014). https://doi.org/10.1007/978-3-662-45563-0_14
55. Tax, N., Lu, X., Sidorova, N., Fahland, D., van der Aalst, W.M.P.: The imprecisions of precision measures in process mining. *Inf. Process. Lett.* **135**, 1–8 (2018)
56. Taymouri, F., Carmona, J.: Model and event log reductions to boost the computation of alignments. In: Ceravolo, P., Guetl, C., Rinderle-Ma, S. (eds.) *SIMPDA 2016. LNBIP*, vol. 307, pp. 1–21. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-74161-1_1
57. Taymouri, F., Carmona, J.: A recursive paradigm for aligning observed behavior of large structured process models. In: *14th International Conference of Business Process Management (BPM)*, Rio de Janeiro, Brazil, 18–22 September 2016
58. Taymouri, F., Carmona, J.: An evolutionary technique to approximate multiple optimal alignments. In: Weske, M., Montali, M., Weber, I., vom Brocke, J. (eds.) *BPM 2018. LNCS*, vol. 11080, pp. 215–232. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-98648-7_13
59. Taymouri, F., Carmona, J.: Model and event log reductions to boost the computation of alignments. In: Ceravolo, P., Guetl, C., Rinderle-Ma, S. (eds.) *SIMPDA 2016. LNBIP*, vol. 307, pp. 1–21. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-74161-1_1

60. Taymouri, F., Carmona, J.: Structural computation of alignments of business processes over partial orders. In: 19th International Conference on Application of Concurrency to System Design, ACSD 2019, Aachen, Germany, 23–28 June 2019, pp. 73–81 (2019)
61. Taymouri, F., Carmona, J.: Computing alignments of well-formed process models using local search. *ACM Trans. Softw. Eng. Methodol.* **29**(3), 15:1–15:41 (2020)
62. Valencia-Parra, Á., Varela-Vaca, Á, J., Teresa Gómez López, M., Carmona, J., Bergenthum, R.: Empowering conformance checking using big data through horizontal decomposition. *Inf. Syst.* **99**, 101731 (2021)
63. van der Aalst, W.M.P.: Decomposing petri nets for process mining: a generic approach. *Distrib. Parallel Databases* **31**(4), 471–507 (2013)
64. van der Aalst, W.M.P., Adriansyah, A., van Dongen, B.F.: Replaying history on process models for conformance checking and performance analysis. *WIREs Data Min. Knowl. Discov.* **2**(2), 182–192 (2012)
65. van der Aalst, W.M.P., Weijters, T., Maruster, L.: Workflow mining: Discovering process models from event logs. *IEEE Trans. Knowl. Data Eng.* **16**(9), 1128–1142 (2004)
66. van Dongen, B., Carmona, J., Chatain, T., Taymouri, F.: Aligning modeled and observed behavior: a compromise between computation complexity and quality. In: Dubois, E., Pohl, K. (eds.) CAiSE 2017. LNCS, vol. 10253, pp. 94–109. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-59536-8_7
67. van Dongen, B.F.: Efficiently computing alignments. In: Daniel, F., Sheng, Q., Motahari, H. (eds.) BPM 2018. LNBIP, vol. 342, pp. 44–55. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-11641-5_4
68. Dongen, B.F.: Efficiently computing alignments. In: Weske, M., Montali, M., Weber, I., vom Brocke, J. (eds.) BPM 2018. LNCS, vol. 11080, pp. 197–214. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-98648-7_12
69. van Dongen, B.F., Carmona, J., Chatain, T.: A unified approach for measuring precision and generalization based on anti-alignments. In: La Rosa, M., Loos, P., Pastor, O. (eds.) BPM 2016. LNCS, vol. 9850, pp. 39–56. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-45348-4_3
70. van Zelst, S.J., Bolt, A., Hassani, M., van Dongen, B.F., van der Aalst, W.M.P.: Online conformance checking: relating event streams to process models using prefix-alignments. *Int. J. Data Sci. Anal.* **8**(3), 269–284 (2019)
71. vanden Broucke, S.K.L.M., Munoz-Gama, J., Carmona, J., Baesens, B., Vanthienen, J.: Event-based real-time decomposed conformance analysis. In: Proceedings on the Move to Meaningful Internet Systems: OTM 2014 Conferences - Confederated International Conferences: CoopIS, and ODBASE 2014, Amantea, Italy, 27–31 October 2014, pp. 345–363 (2014)
72. vanden Broucke, S.K.L.M., De Weerd, J., Vanthienen, J., Baesens, B.: Determining process model precision and generalization with weighted artificial negative events. *IEEE Trans. Knowl. Data Eng.* **26**(8), 1877–1889 (2014)
73. Verbeek, H.M.W., van der Aalst, W.M.P.: Merging alignments for decomposed replay. In: Kordon, F., Moldt, D. (eds.) PETRI NETS 2016. LNCS, vol. 9698, pp. 219–239. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-39086-4_14
74. Wang, J., Song, S., Lin, X., Zhu, X., Pei, J.: Cleaning structured event logs: a graph repair approach. In: Gehrke, J., Lehner, W., Shim, K., Cha, S.K., Lohman, G.M. (eds.) 31st IEEE International Conference on Data Engineering, ICDE 2015, Seoul, South Korea, 13–17 April 2015, pp. 30–41. IEEE Computer Society (2015)

75. Weidlich, M., Mendling, J., Weske, M.: Efficient consistency measurement based on behavioral profiles of process models. *IEEE Trans. Softw. Eng.* **37**(3), 410–429 (2011)
76. Weidlich, M., Polyvyanyy, A., Desai, N., Mendling, J., Weske, M.: Process compliance analysis based on behavioural profiles. *Inf. Syst.* **36**(7), 1009–1025 (2011)
77. Weidlich, M., Polyvyanyy, A., Mendling, J., Weske, M.: Causal behavioural profiles - efficient computation, applications, and evaluation. *Fundam. Informaticae* **113**(3–4), 399–435 (2011)
78. Weidlich, M., Ziekow, H., Mendling, J., Günther, O., Weske, M., Desai, N.: Event-based monitoring of process execution violations. In: Rinderle-Ma, S., Toumani, F., Wolf, K. (eds.) *BPM 2011*. LNCS, vol. 6896, pp. 182–198. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-23059-2_16
79. (Ton) Weijters, A.J.M.M., van der Aalst, W.M.P., Alves De Medeiros, A.K.: Process mining with the heuristics miner-algorithm. Technical Report 166, Technische Universiteit Eindhoven (2006)
80. Zha, H., Wang, J., Wen, L., Wang, C., Sun, J.: A workflow net similarity measure based on transition adjacency relations. *Comput. Ind.* **61**(5), 463–471 (2010)

Open Access This chapter is licensed under the terms of the Creative Commons Attribution 4.0 International License (<http://creativecommons.org/licenses/by/4.0/>), which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons license and indicate if changes were made.

The images or other third party material in this chapter are included in the chapter's Creative Commons license, unless indicated otherwise in a credit line to the material. If material is not included in the chapter's Creative Commons license and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder.

